



UNIVERSITÀ DEGLI STUDI DI FIRENZE

SCUOLA DI INGEGNERIA — DIPARTIMENTO DI INGEGNERIA
DELL'INFORMAZIONE

Elaborato di Tipo A per il corso di
Sistemi Distribuiti

MOBILITY ADVISOR PER SNAP4CITY

Un client MCP con orchestrazione LangGraph
per la pianificazione multimodale di percorsi urbani

Candidato

Junliang Zheng

Matricola 7046609

Docente

Prof. Paolo Nesi

Anno Accademico 2025/2026

Indice

1	Introduzione	1
1.1	Obiettivo	1
1.2	Contenuto della consegna	2
2	Contesto e tecnologie	3
2.1	Model Context Protocol (MCP)	3
2.2	LangGraph	3
2.3	Le sorgenti dati Snap4City e km4city	4
2.4	Il modello linguistico	4
3	Architettura	5
3.1	Il principio: un solo punto di non determinismo	5
3.2	I componenti	6
3.3	Il flusso di un turno	7
4	Implementazione	9
4.1	Estrazione dei parametri	9
4.2	Risoluzione degli estremi del percorso	9
4.3	Calcolo dei percorsi	11
4.4	La geometria reale delle tratte in autobus	12
4.5	Il front-end	12

<i>Indice</i>	ii
5 Distribuzione e integrazione	13
6 Test e validazione	16
6.1 Suite automatica	16
6.2 Validazione sul campo	16
7 Conclusioni	18
Bibliografia	18

Capitolo 1

Introduzione

Snap4City è la piattaforma *Smart City* sviluppata dal DISIT Lab dell'Università degli Studi di Firenze: raccoglie, federa ed espone dati urbani (viabilità, trasporto pubblico, servizi, sensori IoT) attraverso la knowledge base km4city e un insieme di servizi web e dashboard configurabili.

Questo elaborato realizza un **mobility advisor** conversazionale per quella piattaforma: l'utente pone una domanda di viaggio in linguaggio naturale (“*da Piazza del Duomo a Santa Croce*”) e il sistema risponde con il confronto fra i modi di trasporto disponibili — due itinerari **monomodali**, a piedi e in auto, e uno **multimodale** con il trasporto pubblico — disegnando i percorsi su una mappa della dashboard.

1.1 Obiettivo

L'obiettivo è collegare un modello linguistico e un insieme di strumenti remoti esposti secondo il *Model Context Protocol* (MCP) in modo che il risultato sia **affidabile e riproducibile**. Il sistema è realizzato come un **grafo de-**

terministico (Capitolo 3) in cui il modello interviene una sola volta, per estrarre i parametri della richiesta.

1.2 Contenuto della consegna

Il server MCP *advisor* remoto fa parte della piattaforma Snap4City ed è ospitato sulla rete interna del DISIT: non fa parte di questa consegna. Questo progetto fornisce:

- il **client MCP** e l'**orchestratore LangGraph** (`orchestrator.py`);
- un **server MCP locale** (`mcp_server.py`) che espone geocodifica diretta e calcolo percorsi per tutti i modi;
- un **bridge HTTP** FastAPI (`api.py`) fra la dashboard e l'orchestratore;
- il **front-end** della dashboard (`frontend/`), una chat box integrata come `widgetExternalContent`;
- la **suite di test** offline (`tests/`), i diagrammi (`docs/diagrams/`), le schermate (`screenshots/`) e gli output reali (`examples/`).

Capitolo 2

Contesto e tecnologie

2.1 Model Context Protocol (MCP)

MCP è un protocollo aperto che standardizza il modo in cui un'applicazione basata su LLM accede a strumenti e risorse esterne [1]. Un *server* MCP dichiara un insieme di *tool* con schema JSON di ingresso e uscita; un *client* li scopre a runtime e li invoca. In questo progetto il client è realizzato con FastMCP [2] e dialoga con due server distinti:

- il server **remoto** della piattaforma Snap4City (`snap4agentic_advisor_native`), scoperto automaticamente tramite l'endpoint `/apps.json` della dashboard;
- il server **locale** sviluppato in questo elaborato.

2.2 LangGraph

LangGraph [3] modella il flusso applicativo come un grafo di stato: ogni nodo è una funzione che riceve e restituisce porzioni di uno stato tipizzato. Qui

il grafo è volutamente lineare (`understand` $\rightarrow$ `execute` $\rightarrow$ `respond`), senza cicli e senza rami decisi dal modello.

Un vincolo del framework ha guidato la progettazione dello stato: le chiavi restituite da un nodo che **non** sono dichiarate nello schema dello stato vengono silenziosamente scartate. Ogni dato che attraversa i nodi ha quindi un canale esplicito in `AdvisorState`.

2.3 Le sorgenti dati Snap4City e km4city

ServiceMap km4city indice di indirizzi e punti di interesse della Toscana; usato per la geocodifica diretta (testo $\rightarrow$ coordinate).

What-If GraphHopper router motore di routing di Snap4City, con profili `foot`, `car` e `bus`; quest'ultimo alimentato dai feed GTFS regionali.

API *tpl* servizio pubblico km4city che espone linee e geometrie del trasporto pubblico locale.

Server MCP remoto geocodifica inversa, ricerca di servizi per prossimità e disponibilità in tempo reale dei parcheggi.

2.4 Il modello linguistico

Il modello è **Llama 4** servito dall'endpoint Snap4City `llama4-agentic-inference`, compatibile con l'API OpenAI e dotato di *tool calling* nativo. È raggiungibile solo dall'ambiente JupyterHub della piattaforma, con credenziali legate a un account funzionale.

Capitolo 3

Architettura

3.1 Il principio: un solo punto di non determinismo

Il primo prototipo era agentico: il modello riceveva l'elenco dei tool e decideva da sé quali invocare. Si è rivelato inaffidabile. Quando il modello “voleva” raccontare qualcosa, produceva la chiamata a strumento come testo in prosa invece che come struttura `tool_calls`: la chiamata non veniva eseguita e il testo finiva nella risposta finale, con il campo dati vuoto.

La correzione non è stata irrobustire il parser, ma **eliminare il ciclo agentico**. Nel sistema finale:

- `understand` è l'*unica* chiamata al modello, effettuata con `tool_choice` forzato su uno schema di estrazione: il modello non può fare altro che restituire una struttura;
- `execute` è Python puro: applica una sequenza fissa di chiamate MCP;

- **respond** è Python puro: compone la risposta in italiano a partire dai soli dati calcolati.

Anche **respond** usava inizialmente il modello, per “mettere in parole” i dati. È stato rimosso: tutto ciò che produceva era derivabile deterministicamente dai risultati di **execute**, mentre ogni chiamata aggiuntiva raddoppiava l’esposizione ai timeout del gateway e introduceva rischi di deriva linguistica e di invenzione di dati (in un caso il modello aveva elencato numeri di linea inesistenti). Il risultato è una sola chiamata LLM per turno.

3.2 I componenti

La Figura 3.1 mostra la struttura complessiva.

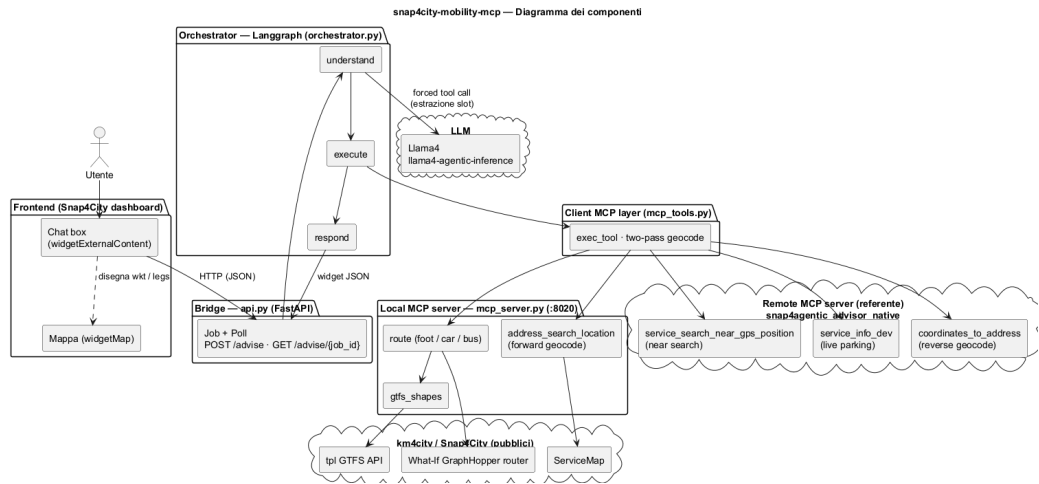


Figura 3.1: Diagramma dei componenti: front-end, bridge, orchestratore, i due server MCP e i servizi esterni.

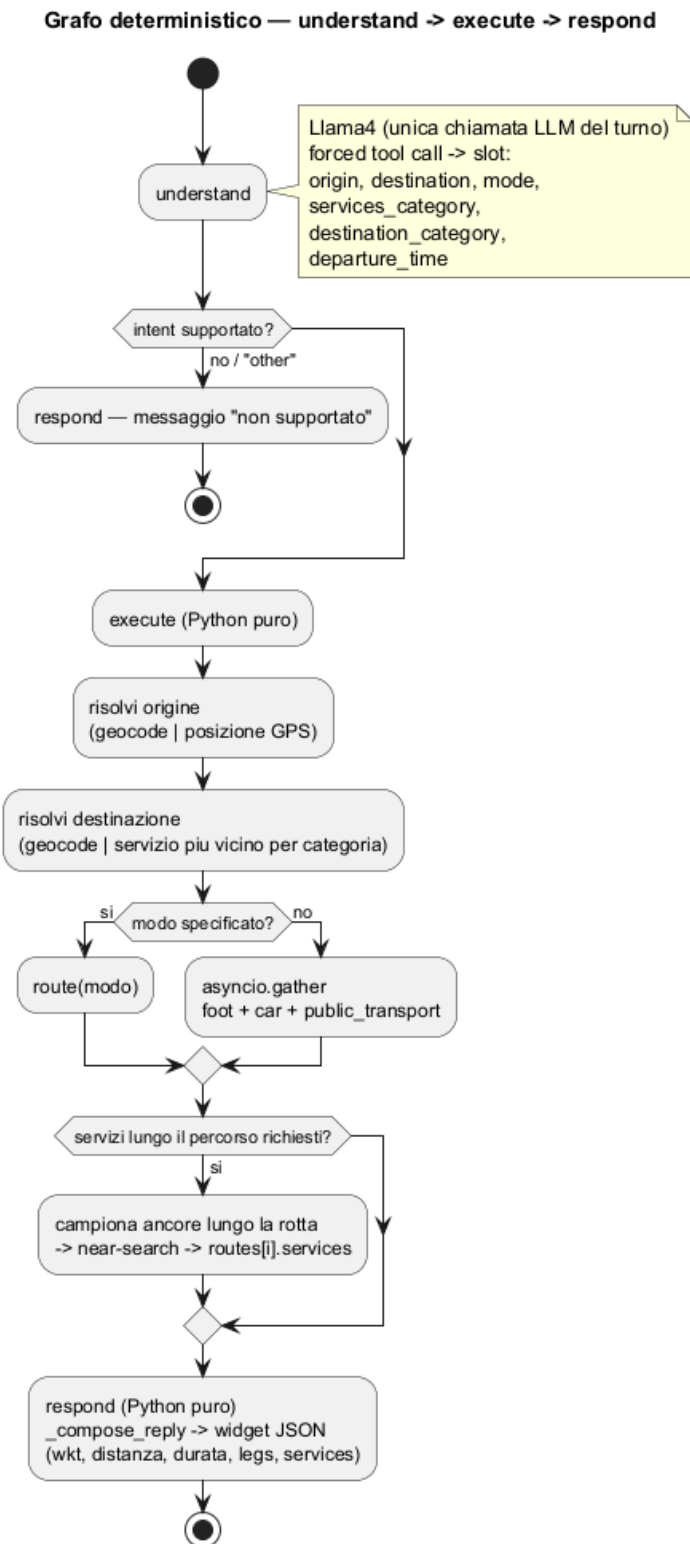
La ripartizione degli strumenti fra i due server MCP non è arbitraria. La geocodifica diretta e il routing sono serviti **localmente** perché i corrispondenti tool remoti non erano utilizzabili:

- il tool remoto `address_search_location` restituisce, per una via fiorentina, zero risultati toscani (i primi sono ad Anversa e in Grecia) e non ordina per punteggio di rilevanza; nessun parametro dello schema permette di correggerlo. Il server locale interroga direttamente la ServiceMap pubblica, che sulla stessa interrogazione restituisce il risultato corretto in prima posizione;
- il tool remoto `routing` non ha mai restituito trasporto pubblico reale e richiedeva una lunga catena di tentativi per gestire risposte vuote transitorie. È stato sostituito dal router What-If.

Restano remoti tre tool che funzionano correttamente: geocodifica inversa (`coordinates_to_address`), ricerca per prossimità (`service_search_near_gps_position`) e disponibilità parcheggi (`service_info_dev`).

3.3 Il flusso di un turno

La Figura 3.2 descrive la logica interna del grafo.

Figura 3.2: Grafo deterministico: il modello interviene solo nel nodo `understand`.

Capitolo 4

Implementazione

4.1 Estrazione dei parametri

Il nodo `understand` invia la domanda dell'utente, insieme agli ultimi turni di conversazione, con una chiamata a strumento forzata. Lo schema estrae: intento, testo dell'origine, testo della destinazione, modo di trasporto, categoria di destinazione (per richieste generiche come “*la farmacia più vicina*”), categoria dei servizi da mostrare lungo il percorso e orario di partenza.

La cronologia inviata al modello è limitata agli ultimi otto messaggi: il contesto per un *follow-up* sta negli ultimi turni, mentre una cronologia illimitata amplifica la latenza e il rischio di errori del gateway.

4.2 Risoluzione degli estremi del percorso

È la parte con più casistica, perché l'indice geografico non offre alcun parametro di vincolo territoriale. La strategia è a scalini:

1. **Doppia passata:** prima si cercano solo indirizzi (`excludePOI=true`), le cui coordinate cadono sul grafo stradale; si ricade sui punti di inte-

resse solo se la prima passata non produce risultati nella città indicata. Senza questo accorgimento “*Piazza del Duomo*” restituisce prima un’azienda omonima.

2. **Città nominata:** se l’utente nomina una città, i candidati vengono ristretti a quella.
3. **Ancoraggio:** altrimenti si sceglie il candidato più vicino a un punto di riferimento — la destinazione si ancora all’origine già risolta, l’origine alla posizione GPS. Senza l’ancoraggio, “*via Pisana 166*” partendo da Firenze veniva risolta a Lucca, con un percorso di 65 km formalmente corretto ma sbagliato nei fatti.
4. **Numero civico:** se il testo contiene un numero civico si privilegia la corrispondenza esatta sul campo `civic`, poi le etichette di forma stradale, e solo da ultimo la vicinanza all’ancora. Le interrogazioni senza numero civico non sono toccate da questo scalino.
5. **Reinterrogazione con la città dell’ancora:** l’indice ordina per sola rilevanza testuale e tronca i risultati, quindi la via omonima della città dell’utente può non comparire affatto fra i candidati. Se l’utente non ha nominato una città, la ricerca viene ripetuta aggiungendo il nome della città dedotta. Un controllo sui *token* significativi evita che un punto di riferimento di un’altra città (“*Piazza dei Miracoli*”) venga attirato erroneamente.

Se l’origine manca del tutto, si usa la posizione GPS del browser, geocodificata a ritroso una sola volta per poter dire “*dalla tua posizione*”; in assenza di GPS il sistema chiede il punto di partenza invece di indovinare.

4.3 Calcolo dei percorsi

Quando l'utente non indica un modo, i tre profili vengono calcolati **concorrentemente** con un unico `asyncio.gather` piatto, e il turno risponde una sola volta quando tutti sono pronti: il tempo totale è quello del più lento, non la somma. Se il modo è indicato esplicitamente si calcola solo quello, in modo che chi chiede un percorso a piedi non paghi la latenza del trasporto pubblico.

I tre risultati non hanno però la stessa struttura. A piedi e in auto sono itinerari **monomodali**: un unico mezzo copre l'intero tragitto, dall'origine alla destinazione. Quello con il trasporto pubblico è invece **multimodale**, perché combina un tratto pedonale fino alla fermata di salita, una o più tratte a bordo dell'autobus e un tratto pedonale finale. Per questo il back-end lo restituisce suddiviso in tratte (**legs**), ciascuna con la propria geometria e il proprio mezzo, mentre per un itinerario monomodale è sufficiente un'unica geometria: è la suddivisione che permette al front-end di disegnare con colori distinti la parte a piedi e quella in autobus, e di collocare i segnaposto alle fermate di salita e di discesa.

L'eventuale orario di partenza viene passato *solo* al profilo di trasporto pubblico, l'unico con un modello temporale (gli orari GTFS); il fuso orario viene fissato esplicitamente a **Europe/Rome**, perché il servlet interpreta le date prive di offset nel proprio fuso e il processo sul JupyterHub lavora in UTC.

Un itinerario di trasporto pubblico che il router restituisce interamente pedonale non è un errore: su tragitti brevi camminare domina qualunque attesa. In quel caso il percorso perde la propria natura multimodale e viene ri-etichettato come pedonale, presentato onestamente come tale.

4.4 La geometria reale delle tratte in autobus

Per una tratta percorsa in autobus il router restituisce **un vertice per fermata**: la geometria reale del percorso non è presente nella sua risposta, quindi la linea disegnata taglia gli isolati in linea retta.

Il modulo `gtfs_shapes.py` la recupera a runtime dall'API pubblica *tpl* di `km4city`: individua la variante di linea che corrisponde alla tratta e ne ritaglia il tratto compreso fra la fermata di salita e quella di discesa. Quando nessuna variante corrisponde, la tratta conserva la geometria originale: la funzionalità degrada, non fallisce.

4.5 Il front-end

La chat box è un `widgetExternalContent` incollato nella dashboard, che disegna i percorsi su un `widgetMap` adiacente. Il widget dispone di un ramo *manual* che collega semplicemente i punti forniti: usandolo, la mappa **non interroga alcun router** e la linea compare insieme al testo della risposta. Per l'itinerario multimodale il back-end invia la suddivisione in tratte a piedi e in autobus, che vengono disegnate con colori distinti e con i segnaposto alle fermate di salita e discesa.

Quando un turno restituisce più percorsi, una barra di *chip* permette di selezionarne uno: la selezione è **puramente locale** — ridisegna la mappa, mostra il dettaglio già pre-calcolato dal back-end e commuta i segnaposto — perché un nuovo turno costerebbe di nuovo la latenza dell'autobus. Nella cronologia conversazionale viene riscritta solo una riga di eco della scelta, sufficiente a mantenere il contesto per le domande successive senza gonfiare i prompt seguenti.

Capitolo 5

Distribuzione e integrazione

Il sistema gira sulla **JupyterHub di Snap4City**, unico ambiente da cui sono raggiungibili sia il server MCP remoto (rete interna) sia l'endpoint del modello. Come mostra la Figura 5.1, servono due processi: il server MCP locale sulla porta 8020 e il bridge FastAPI sulla porta 8010. Il browser raggiunge il bridge *same-origin* attraverso `jupyter-server-proxy`.

La Figura 5.2 mostra il risultato finale nella dashboard: un turno senza modo indicato produce i tre percorsi, il testo di confronto e la barra di selezione.

La Figura 5.3 mostra la funzionalità dei servizi lungo il percorso. Poiché lo strumento remoto di ricerca accetta solo un punto e un raggio, il corridoio viene costruito lato client campionando ancora lungo la geometria del percorso ed eseguendo una ricerca attorno a ciascuna, con successiva deduplicazione dei risultati.

Le restanti schermate — selezione delle singole modalità tramite le chip, richieste a modo singolo e destinazione per categoria a partire dalla posizione GPS — sono raccolte nella cartella `screenshots/` e descritte nel relativo indice.

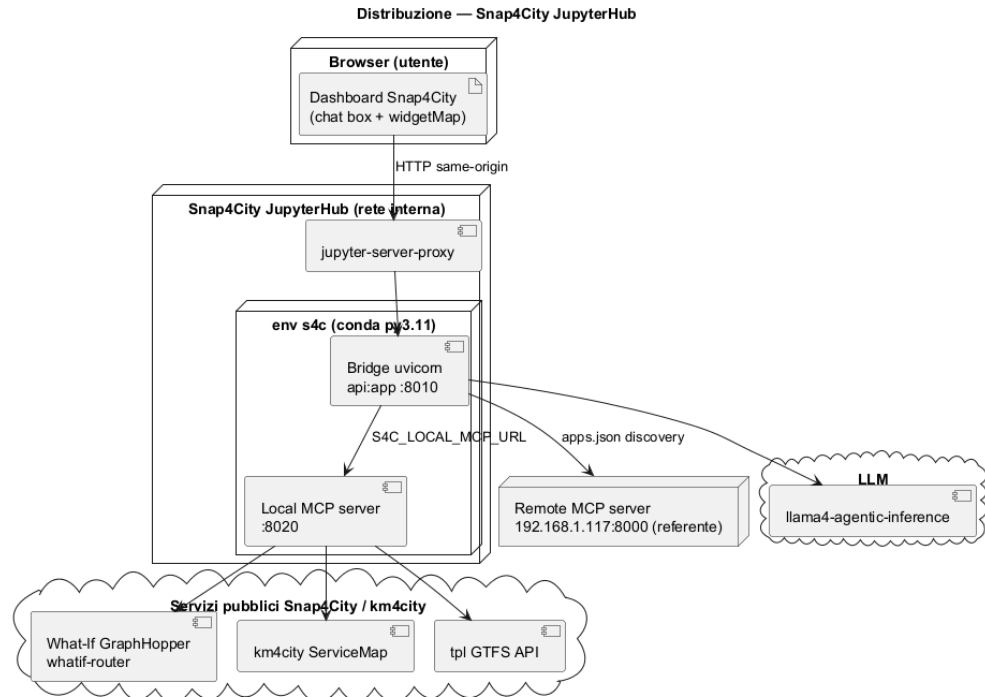


Figura 5.1: Distribuzione: due processi nell'ambiente JupyterHub, la dashboard nel browser, i servizi esterni.

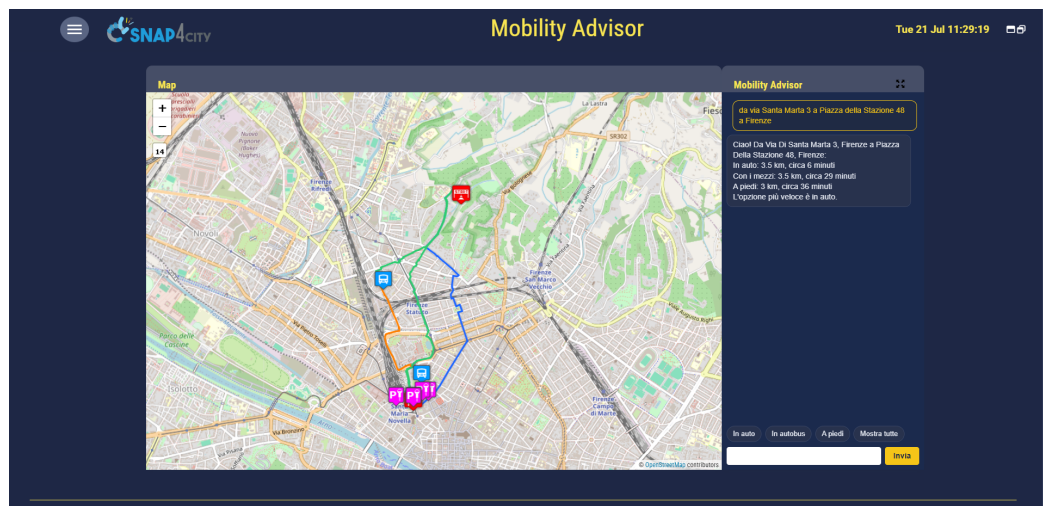


Figura 5.2: Turno a tre modi: i due itinerari monomodali (a piedi, in auto) e quello multimodale (trasporto pubblico) disegnati insieme sulla mappa della dashboard.

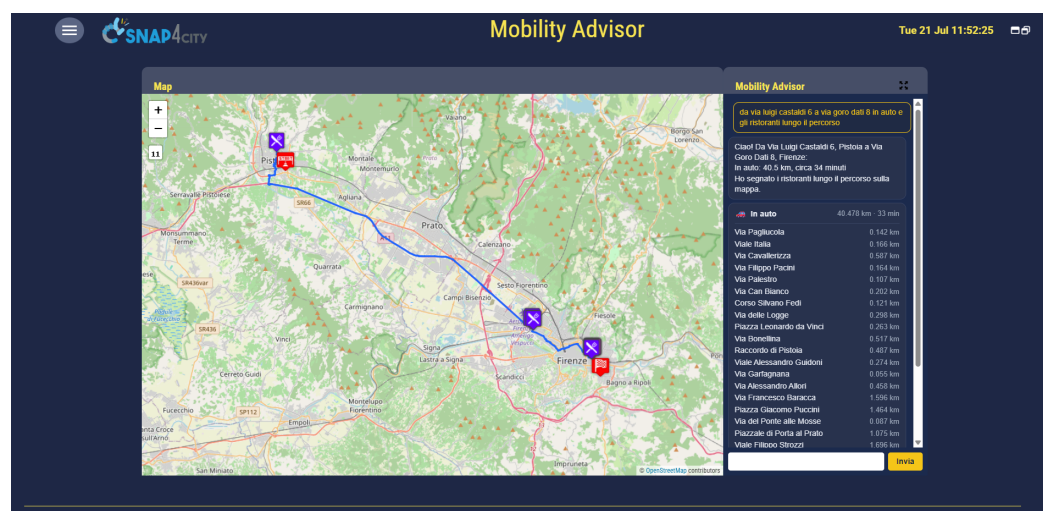


Figura 5.3: Servizi della categoria richiesta individuati lungo il percorso.

Capitolo 6

Test e validazione

6.1 Suite automatica

La suite conta **126 test** eseguibili ovunque: sono completamente *offline* e non richiedono né il modello né i server MCP. I doppi di test sostituiscono il client MCP con una coda deterministica di risposte e il client LLM con una risposta predefinita; la Tabella 6.1 ne riassume la composizione.

6.2 Validazione sul campo

Il sistema è stato verificato end-to-end sulla dashboard. La Tabella 6.2 riporta i valori di alcuni turni reali, i cui output completi sono allegati nella cartella `examples/`.

Il confronto a tre modi è coerente con l'intuizione: l'auto è la più rapida, il percorso pedonale è il più breve in distanza ma il più lento, il trasporto pubblico si colloca in mezzo. Sul fronte dei tempi di risposta, le misure sul router mostrano una netta asimmetria: i profili `foot` e `car` rispondono in circa 0,3–0,5 secondi, mentre `bus` richiede 30–45 secondi.

File	Test	Copertura
<code>test_orchestrator.py</code>	74	nodi del grafo, risoluzione degli estremi, concorrenza, composizione della risposta
<code>test_mcp_server.py</code>	20	tool <code>route</code> , suddivisione delle tratte, geocodifica
<code>test_mcp_tools.py</code>	13	doppia passata, parser degli envelope, riduzione del contesto
<code>test_api.py</code>	9	sanificazione del GPS e API del bridge
<code>test_gtfs_shapes.py</code>	6	scelta della variante, taglio della geometria, degradazioni
<code>test_llm.py</code>	4	ritentativi su errori transitori, credenziali

Tabella 6.1: Composizione della suite di test.

Scenario	Modo	Distanza	Durata
Turno a tre modi	auto	2,872 km	0:04:07
	trasporto pubblico	2,715 km	0:16:59
	a piedi	2,010 km	0:24:07
Tragitto più lungo	trasporto pubblico	6,936 km	0:32:10
Auto + servizi lungo il percorso	auto	40,478 km	0:33:58

Tabella 6.2: Misure da turni reali (output completi in `examples/`).

Capitolo 7

Conclusioni

Il lavoro ha prodotto un advisor di mobilità funzionante e integrato nella dashboard Snap4City, capace di rispondere in linguaggio naturale confrontando percorsi a piedi, in auto e con il trasporto pubblico, di risolvere destinazioni generiche a partire dalla posizione dell'utente e di mostrare i servizi lungo il tragitto.

L'architettura scelta — una sola chiamata al modello linguistico, per l'estrazione dei parametri, e codice deterministico per l'esecuzione degli strumenti e la formulazione della risposta — rende il comportamento del sistema riproducibile e verificabile con una suite di test interamente offline.

Bibliografia

- [1] Anthropic, *Model Context Protocol — Specification*. <https://modelcontextprotocol.io>
- [2] *FastMCP — The fast, Pythonic way to build MCP servers and clients*. <https://github.com/jlowin/fastmcp>
- [3] LangChain, *LangGraph — Building stateful, multi-actor applications with LLMs*. <https://langchain-ai.github.io/langgraph/>
- [4] DISIT Lab, Università degli Studi di Firenze, *Snap4City — Smart aNalytic APp builder for sentient Cities*. <https://www.snap4city.org>
- [5] DISIT Lab, *Km4City — Knowledge Model for the City*. <https://www.km4city.org>
- [6] GraphHopper GmbH, *GraphHopper Routing Engine*. <https://www.graphhopper.com>
- [7] *General Transit Feed Specification (GTFS) Reference*. <https://gtfs.org/documentation/schedule/reference/>